

15-2. 最长回文子序列

回文(palindrome)是正序与逆序相同的非空字符串。例如, 所有长度为 1 的字符串、civic、racecar、aibohphobia 都是回文。设计高效算法, 求给定输入字符串的最长回文子序列。例如, 给定输入 character, 算法应该返回 carac。算法的运行时间是怎样的?

答案: 对任意字符串, 如果头和尾相同, 那么它的最长回文子序列一定是去头去尾之后的部分的最长回文子序列加上头和尾。如果头和尾不同, 那么它的最长回文子序列是去头的部分的最长回文子序列和去尾的部分的最长回文子序列的较长的那一个。

设字符串为 s , $f(i,j)$ 表示 $s[i..j]$ 的最长回文子序列。则有

$$f(i,j) = \begin{cases} 2 + f(i+1, j-1), & \text{if } s[i] = s[j] \\ \max\{f(i, j-1), f(i+1, j)\}, & \text{if } s[i] \neq s[j] \end{cases}$$
$$f(i,j) = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i > j \end{cases}$$

由于 $f(i,j)$ 依赖 $i+1$, 所以循环计算的时候, 第一维必须倒过来计算, 从 $s.length()-1$ 到 0。

最后, s 的最长回文子序列长度为 $f(0, s.length()-1)$ 。

15-4. 整齐打印

考虑在一个打印机上整齐地打印一段文章的问题。输入的正文是 n 个长度分别为 $l_1, l_2, \dots, l_n$ (以字符个数度量) 的单词构成的序列。我们希望将这个段落落在一些行上整齐地打印出来, 每行至多 M 个字符。“整齐”的标准如下: 如果某一行包含从 i 到 j 的单词 ($j \geq i$), 且单词之间只留一个空格, 则在行末多余的空格字符个数为 $M - j + i - \sum_{k=i}^j l_k$, 它必须是非负值才能让该行容纳这些单词。我们希望所有行 (除最后一行) 的行末多余空格字符个数的立方和最小。请给出一个动态规划的算法, 来在打印机整齐地打印一段又 n 个单词的文章。分析所给算法的执行时间和空间需求。

答案: 构建数组 $a[i]$ 代表包含 $1 \rightarrow i$ 单词的最优“整齐度”——空格的立方总和。开始时, 令 $a[0]=0$ 。 $a[i]$ 的递推式如下:

$$a[i] = \min_{0 \leq j \leq i} (a[j] + W[j][i])$$

$W[j][i]$ 代表将 j 到 i 的单词写入一行时, 该行的多余空格数的立方。

$$W[j][i] = (M - j + i - \sum_{k=j}^i l_k)^3$$

每次计算 $a[i]$ 时, 循环 j 从 $i-1$ 开始, 直到某个 j 值使得值为负, 则跳出。

最终 $a[n]$ 即为所求的最小立方总和。

16-1. 找零问题

考虑用最少的硬币找 n 美分零钱的问题。假设每种硬币的面额都是整数。

- a) 设计贪心算法求解找零问题, 假定有 25 美分、10 美分、5 美分和 1 美分 4 种面额的硬币。证明你的算法能找到最优解。

答案: 若 n 是正整数, 则用 25 美分、10 美分、5 美分和 1 美分等尽可能少的硬币找出的 n 美分零钱中, 至多有 2 个 10 美分、至多有 1 个 5 美分、至多有 4 个 1 美分硬币, 而不能有 2 个 10 美分和 1 个 5 美分硬币。用 10 美分、5 美分和 1 美分硬币找出的零钱不能超过 24 美分。

证用反证法。证明如果有超过规定数目的各种类型的硬币, 就可以用等值的数目更少的硬币来替换。注意, 如果有 3 个 10 美分硬币, 就可以换成 1 个 25 美分和 1 个 5 美分硬币; 如果有 2 个 5 美分硬币, 就可以换成 1 个 10 美分硬币; 如果有 5 个 1 美分硬币, 就可以换成 1 个 5 美分硬币; 如果有 2 个 10 美分和 1 个 5 美分硬币, 就可以换成 1 个 25 美分硬币。由于至多可以有 2 个 10 美分、1 个 5 美分和 4 个 1 美分硬币, 而不能有 2 个 10 美分和 1 个 5 美分硬币, 所以当用尽可能少的硬币找 n 美分零钱时, 24 美分就是用 10 美分、5 美分和 1 美分硬币能找出的最大值。

假设存在正整数 n , 使得有办法将 25 美分、10 美分、5 美分和 1 美分硬币用少于贪心算法所求出的硬币去找 n 美分零钱。首先注意, 在这种找 n 美分零钱的最优方式中使用 25 美分硬币的个数 q' , 一定等于贪心算法所用 25 美分硬币的个数。为说明这一点, 注意贪心算法使用尽可能多的 25 美分硬币, 所以 $q' \leq q$ 。但是 q' 也不能小于 q 。假如 q' 小于 q , 需要在这种最优方式中用 10 美分、5 美分和 1 美分硬币至少找出 25 美分零钱。而根据引理 1, 这是不可能的。由于在找零钱的这两种方式中一定有同样多的 25 美分硬币, 所以在这两种方式中 10 美分、5 美分和 1 美分硬币的总值一定相等, 并且这些硬币的总值不超过 24 美分。10 美分硬币的个数一定相等, 因为贪心算法使用尽可能多的 10 美分硬币。而根据引理 1, 当使用尽可能少的硬币找零钱时, 至多使用 1 个 5 分硬币和 4 个 1 分硬币, 所以在找零钱的最优方式中也使用尽可能多的 10 美分硬币。类似地, 5 美分硬币的个数相等; 最终, 1 美分的个数相等。

b) 假定硬币的面额是 c 的幂, 即面额 $c^0, c^1, \dots, c^k$, c 和 k 为整数, $c > 1$, $k \geq 1$ 。证明: 贪心算法总能得到最优解。

答案: 同 a 问, 由于 $c^0 + c^1 + \dots + c^{k-1} = c^k - 1 < c^k$, 故当 n 大于 c^k 时, 可以分解为 c^k 与 $n - c^k$ 的值, 其中 c^k 只用一个硬币值为 c^k 的硬币就能得到最少硬币数, 而子问题变成 $n - c^k$ 的最少硬币数, 依次类推, 贪心算法总能得到最好的结果。

c) 设计一组硬币面额, 使得贪心算法不能保证得到最优解。这组硬币面额中应该包含 1 美分, 使得对每个零钱值都存在找零方案。

答案: 如果出现一组硬币 6, 5, 1。当遇到 10 分时, 按照贪心算法将分解为 $6 + 4 \times 1$, 而其实为 2×5 。

d) 设计一个 $O(nk)$ 时间的找零算法, 适用于任何 k 种不同面额的硬币, 假定问题包含 1 美分硬币。

答案: 假设 $m[n]$ 表示找零 n 美分需要的最少硬币数, 硬币面值为 $c^0, c^1, \dots, c^k$, 并令 $m[0] = 0$, 则 $m[n] = 1$ 如果 n 等于某个 c^i , 否则 $m[n] = \min\{m(n - c^1) + 1, m(n - c^2) + 1, \dots, m(n - c^k) + 1\}$ 。基于此公式, 可以设计动态规划算法。

16-2. 最小平均完成时间调度问题

假定给定任务集合 $S = \{a_1, a_2, \dots, a_n\}$, 其中任务 a_i 在启动后需要 p_i 个时间单位完成。你有一台计算机来运行这些任务, 每个时刻只能运行一个任务。令 c_i 表示任务 a_i 的完成时间, 即任务 a_i 执行完的时间。你的目标是最小化平均完成时间, 即最小化 $(1/n) \sum_{i=1}^n c_i$ 。例如, 假定有两个任务 a_1 和 a_2 , $p_1=3$, $p_2=5$, 如果 a_2 首先运行, 然后运行 a_1 , 则 $c_2=5$, $c_1=8$, 平均完成时间为 $(5+8)/2=6.5$ 。如果 a_1 先于 a_2 执行, 则 $c_1=3$, $c_2=8$, 平均完成时间为 $(3+8)/2=5.5$ 。

a. 设计算法, 求平均完成时间最小的调度方案。任务的执行都是非抢占的, 即一旦 a_i 开始运行, 它就持续运行 p_i 个时间单位。证明你的算法能最小化平均完成时间, 并分析算法的运行时间。

b. 现在假定任务并不是在任意时刻都可以开始执行, 每个任务都有一个释放时间 r_i , 在此时间之后才可以开始。此外假定任务执行是可以抢占的 (preemption), 这样任务可以被挂起, 然后再重新开始。例如, 一个任务 a_i 运行时间 $p_i=6$, 释放时间 $r_i=1$, 它可能在时刻 1 开

始运行，在时刻 4 被抢占。然后在时刻 10 恢复运行，在时刻 11 再次被抢占，最后在时刻 13 恢复运行，在时刻 15 运行完毕。任务 a_i 共运行了 6 个时间单位，但运行时间被分割成三部分。在此情况下， a_i 的完成时间为 15。设计算法，对此问题求解平均运行时间最小的调度方案。证明你的算法确实能最小化完成时间，分析算法的运行时间。

答案：a:

按照从最小到最大的处理时间对任务进行排序，并按该顺序运行它们。要知道这个贪心解是最优的，首先要观察这个问题表现出最优的子结构：如果我们在一个最优解中运行第一个任务，那么我们通过以最小化平均完成时间的方式运行剩下的任务来获得一个最优解。设 o 为最优解。设 a 为处理时间最小的任务， b 为在 o 中运行的第一个任务。设 G 为我们在 o 中运行 a 和 b 的顺序转换得到的解。这就通过 a 和 b 处理时间的差异，减少了在 a 和 b 之间 a 的完成时间和 G 中所有任务的完成时间。由于其他所有的补全时间保持不变， G 的平均补全时间小于或等于 o 的平均补全时间，证明贪心解是最优解。运行时间复杂度为 $O(n \lg n)$ ，因为我们必须首先对元素排序。

b:

在不失一般性的前提下，我们假定每个任务都是一个单位时间任务。应用与第(a)部分相同的策略，除非这次我们想要添加到计划旁边的任务还不允许运行，否则我们必须跳过它。由于可能有许多处理时间短但发布时间较晚的任务，所以运行时间复杂度变为 $O(n^2)$ ，因为我们可能要花费 $O(n)$ 个时间来决定下一步添加哪个任务。

22-1. 以广度优先搜索来对图的边进行分类

深度优先搜索将图中的边分类为树边、后向边、前向边和横向边。广度优先搜索也可以用来进行这种分类。具体来说，广度优先搜索将从源结点可以到达的边划分为同样的 4 种类型。

a. 证明在对无向图进行的广度优先搜索中，下面的性质成立：

1. 不存在后向边，也不存在前向边。
2. 对于每条树边 (u, v) ，我们有 $v.d = u.d + 1$ 。
3. 对于每条横向边 (u, v) ，我们有 $v.d = u.d$ 或 $v.d = u.d + 1$ 。

b. 证明在对有向图进行广度优先搜索时，下面的性质成立：

1. 不存在前向边。

2.对于每条树边 (u, v) , 我们有 $v.d = u.d + 1$ 。

3.对于每条横向边 (u, v) , 我们有 $v.d \leq u.d + 1$ 。

4.对于每条后向边 (u, v) , 我们有 $0 \leq v.d \leq u.d$ 。

答案: a:

1.假如 (u, v) 是前向边, 则搜索结点 v 前必搜索 u , 则根据 BFS, 当搜索结点 u 以后必先搜索结点 v , 则 (u, v) 是树边; 同理, 若 (u, v) 是后向边, 则 (v, u) 是树边; 矛盾, 所以不存在前向边和后向边。

2.对于每条树边 (u, v) 有 $v.\pi = u$, 切执行 $v.\pi = u$ 的同时执行 $v.d = u.d + 1$; 在这之后 $u.d$ 和 $v.d$ 都不会改变, 所以 $v.d = u.d + 1$; 得证。

3.由 (u, v) 为横向边可知, 当搜索结点 u 时, v 必须在队列中, 否则 (u, v) 为树边, 所以 $v.d \leq u.d + 1$ 。又由无向图横向边可知 $v.d \geq u.d$ 。所以 $u.d = v.d$ 或 $u.d + 1 = v.d$ 。

b:

1.假如 (u, v) 是前向边, 则 $u.d < v.d$, 则搜索结点 u 时, 结点 v 仍是白色, 则 (u, v) 必是树边, 矛盾; 所以不存在前向边。

2.同 a.2。

3.和 a.3 类似。

4.显然有 $v.d \geq 0$, 又由后向边可知 $v.d \leq u.d$, 得证。

22-3. 欧拉回路

强连通有向图 $G=(V, E)$ 中的一个欧拉回路是指一条遍历图 G 中每条边恰好一次的环路。不过, 这条环路可以多次访问同一个点。

a、证明: 图 G 中有一条欧拉回路当且经当对于图中的每个点 v , 有 v 的入度等于出度。

答案:

b、给出一个复杂度为 $O(E)$ 的算法找出 G 中的一条欧拉回路。(提示: 对边不相交环路进行归并)

答案: 从任意一个起始点 v 开始遍历, 直到再次到达点 v , 即寻找一个环, 这会保证一定可以到达点 v , 因为遍历到任意一个点 u , 由于其出度和入度相同, 故 u 一定存在一条出边, 所以一定可以到达 v 。将此环定义为 C , 如果环 C 中存在某个点 x , 其有出边不在环中, 则继续以此点 x 开始遍历寻找环 C' , 将环 C 、 C' 连接起来也是一个大环, 如此往复, 直到图 G 中所有的边均已经添加到环中。

数据结构如下:

- (1) 使用循环链表 CList 存储当前已经发现的环;
- (2) 使用一个链表 L 保存当前环中还有出边的点;
- (3) 使用邻接表存储图 G

使用如下的步骤可以确保算法的复杂度为 $O(E)$:

- (1) 将图 G 中所有点入 L, 取 L 的第一个结点
- (2) 直接取其邻接表的第一条边, 如此循环往复直到再次到达点 v 构成环 C, 此过程中将 L 中无出边的点删除。环 C 与环 CList 合并, 只要将 CList 中的点 v 使用环 C 代替即可。
- (3) 如果链表 L 为空表示欧拉回路过程结束, 否则取 L 的第一个结点, 继续步骤(2)

23-3. 瓶颈生成树

一个无向图 G 上的瓶颈生成树是 G 上一种特殊的生成树。一个瓶颈生成树 T 上权重最大边的权重是 G 中所有生成树中最小的。T 上最大权重的边的权重称为 T 的值。

a. 证明一个最小生成树是瓶颈生成树。

答案: 使用替换法, 假设瓶颈树 T 的最大边长为 W, 而某棵 MST 的最大边 $> W$, 则将 MST 从最大边切断, 变成两个部分, 选择 T 中连接两部分的边, 则得到更小的树。

a 问说明了求瓶颈生成树不难于求最小生成树。在余下的部分, 我们来设计线性时间算法求解瓶颈生成树。

b. 给定无向图 G 和整数 b, 判定 G 中是否有一颗瓶颈生成树 T, T 上权重最大的边的权重小于等于 b。

答案: DFS 或 BFS 遍历图 G, 跳过所有权值大于 b 的边, 最后若有节点未遍历到, 则 T 值大于 b, 否则不超过 b

c. 结合 b 问设计的算法设计一个线性算法来求解瓶颈生成树问题。

答案: 1. 求出边权值的中位数 (类似于求 nth element 一类问题) M, 以此将图 G 的边按权值分成两部分, 一部分小于等于 M, 另一部分大于 M;

2. 利用 b 提出的方法判断图 G 瓶颈生成树的 T 值是否不超过 M, 也就是看这个 T 值位于大小哪半边;

3. 若位于小半边, 则将大半边里的边删除, 并回到步骤 1;

4. 若位于大半边, 则小半边组成的图必不连通, 将其连通分量各收缩成一个点, 再和大半边重新组成一个图 G2, 并回到步骤 1。

24-1. Yen 对 Bellman-Ford 算法的改进

假设对于 Bellman-Ford 算法按照如下顺序进行松弛操作。为此，我们首先给输入的图 $G = (V, E)$ 上所有点任意指定一个顺序 $v_1, v_2, \dots, v_{|V|}$ ，于是 E 上所有的边就可以分为两类 $E_f = \{(v_i, v_j) : i < j\}$ and $E_b = \{(v_i, v_j) : i > j\}$ ，进而形成 G 的两个子图 $G_f = (V, E_f)$ 和 $G_b = (V, E_b)$ 。注意，这里假设 G 中没有点自环，即没有一条边两个端点是一个点。

- a) 证明 G_f 和 G_b 都是无环的，而且拓扑排序结果分别为 $\langle v_1, v_2, \dots, v_{|V|} \rangle$ 和

$$\langle v_{|V|}, v_{|V|-1}, \dots, v_1 \rangle$$

假设对于 Bellman-Ford 算法按照如下方式进行每一轮的松弛操作。我们首先对 E_f 中的边按照其起点在 $v_1, v_2, \dots, v_{|V|}$ 中的顺序依次进行松弛，然后对 E_b 中的边按照其起点在 $v_{|V|}, v_{|V|-1}, \dots, v_1$ 中的顺序依次进行松弛

- b) 证明在上述算法中，如果 G 中没有由 s 可达的负权环路，那么在 $\lceil |V|/2 \rceil$ 循环之后对 V 中所有点都有 $v.d = \delta(s, v)$
- c) 上述算法是否改善了 Bellman-Ford 算法的渐近运行时间？

答案: a:

反证法。证明假设 G_f 不是无环的是矛盾的。

一个循环必须至少有一条边 (u, v) 其中 u 的下标大于 v 。这条边不在 E_f 中(根据 E_f 的定义)，与 G_f 有一个周期的假设相反。因此 G_f 是没有环的。对 G_f 来说， $\langle v_1, v_2, \dots, v_{|V|} \rangle$ 是一个拓扑排序，因为根据 E_f 的定义，我们知道所有的边都是从较小的下标指向较大的下标。 E_b 的证明与上面相似。

b:

对于所有的边 $v \in V$ ，我们知道要么 $\delta(s, v) = \infty$ 是有限的，要么 $\delta(s, v)$ 是有限的。如果 $\delta(s, v) = \infty$ ，那么 $v.d$ 将是 ∞ 的。因此，我们仅仅需要考虑 $v.d$ 是有限的情况。从 s 到 v 一定有最短路径。假设 $p = \langle v_0, v_1, \dots, v_{k-1}, v_k \rangle$ 是那条最短路径，其中 $v_0 = s$ ， $v_k = v$ 。现在我们考虑 p 方向改变了多少次，这是， $(v_{i-1}, v_i) \in E_f$ 和 $(v_i, v_{i+1}) \in E_b$ ，反之亦然。 p 中最

多有 $|V| - 1$ 条边，所以最多有 $|V| - 2$ 个方向的变化。路径的任何部分没有改变方向计算正确的 d 值在第一或第二个一半的一次单程的顶点开始在没有改变方向序列正确的 d 值,因为边缘是放松的方向序列的顺序。每个方向的改变都需要在路径的新方向上进行半遍。下表根据 $|V| - 1$ 的奇偶性和第一个边的方向显示了所需的最大遍历数:

$ V - 1$	first edge direction	passes
even	forward	$(V - 1)/2$
even	backward	$(V - 1)/2 + 1$
odd	forward	$ V /2$
odd	backward	$ V /2$

在任何情况下,我们需要通过的最大数量是 $\lceil |V|/2 \rceil$ 。

c:

这个计划不会影响算法的渐近运行时间，因为我们只进行 $\lceil |V|/2 \rceil$ 传递，而不执行 $|V| - 1$ ，所以仍然是 $O(V)$ 通道。每个传递仍然需要 $\Theta(E)$ ，所以运行时间仍然是 $O(VE)$ 。